

Stage 2.6 — Automatically Discover and Load All YAML Configuration Files

Moving from manual configuration listing to dynamic directory scanning

Perfect. Let's implement Stage 2.6 — Automatically Discover and Load All YAML Configuration Files. This is a meaningful step because we're moving from:

Manually specify every YAML file ↓ Automatically discover YAML files

1. Current structure

Make sure your project looks like this:

```
yaml-learning/  
├── config/  
│   ├── app.yaml  
│   ├── llm.yaml  
│   ├── embedding.yaml  
│   ├── vector_database.yaml  
│   └── retrieval.yaml  
└── src/  
    ├── yaml_loader.py  
    └── load_config.py
```

Your existing `yaml_loader.py` can remain as it is.

2. The new idea

Previously we did this:

```
app_config = load_yaml(CONFIG_DIR / "app.yaml")  
llm_config = load_yaml(CONFIG_DIR / "llm.yaml")  
embedding_config = load_yaml(CONFIG_DIR / "embedding.yaml")
```

That's manual.

Instead, Python can look inside the config directory and find:

```
*.yaml
```

We can use:

```
CONFIG_DIR.glob("*.yaml")
```

3. Understand Path.glob()

Suppose:

```
CONFIG_DIR
```

points to:

```
yaml-learning/config/
```

Then:

```
CONFIG_DIR.glob("*.yaml")
```

means:

Find all files in this directory whose names end with `.yaml`.

It could return:

```
app.yaml
llm.yaml
embedding.yaml
vector_database.yaml
retrieval.yaml
```

This is exactly what we need.

4. Create load_all_configs()

Open:

```
src/load_config.py
```

Replace the manual loading code with:

```

from pathlib import Path

from yaml_loader import load_yaml

BASE_DIR = Path(__file__).resolve().parent.parent
CONFIG_DIR = BASE_DIR / "config"

def load_all_configs(config_dir):
    configs = {}

    for config_file in config_dir.glob("*.yaml"):
        config = load_yaml(config_file)
        configs.update(config)

    return configs

config = load_all_configs(CONFIG_DIR)

print(config["application"]["name"])
print(config["llm"]["model"])
print(config["embedding"]["model"])
print(config["vector_database"]["collection_name"])
print(config["retrieval"]["top_k"])

```

Run:

```
python src/load_config.py
```

You should get:

```

Enterprise RAG application
llama-3.1-8b-instant
all-MiniLM-L6-v2
employee_handbook
5

```



You've now automatically discovered and loaded all your YAML files.

5. Let's understand this carefully

The heart of our implementation is:

```
for config_file in config_dir.glob("*.yaml"):
```

Imagine your directory contains:

```
config/
├─ app.yaml
├─ llm.yaml
├─ embedding.yaml
├─ vector_database.yaml
└─ retrieval.yaml
```

The loop essentially does:

```
config_dir.glob("*.yaml") | ▼ app.yaml llm.yaml embedding.yaml vector_database.yaml retrieval.yaml | ▼
for each file | ▼ load_yaml()
```

So if tomorrow you add:

```
chunking.yaml
```

you don't need to modify `load_config.py`.

It will automatically discover it.

That's a major improvement.

6. Understand configs = {}

We start with an empty dictionary:

```
configs = {}
```

Then each YAML file is loaded.

For example:

First iteration

`app.yaml`

produces:

```
{
  "application": {
    "name": "Enterprise RAG application",
    ...
  }
}
```

Then:

```
configs.update(config)
```

makes:

```
configs = {
  "application": {
    ...
  }
}
```

Second iteration

llm.yaml produces:

```
{
  "llm": {
    "provider": "Groq",
    "model": "llama-3.1-8b-instant"
  }
}
```

After:

```
configs.update(config)
```

we have:

```
configs = {
  "application": {...},
  "llm": {...}
}
```

Then embedding:

```
application
llm
embedding
```

Then:

```
application
llm
embedding
vector_database
```

Finally:

```
application
llm
embedding
vector_database
retrieval
```

7. Why update() works nicely here

Your YAML files have different root keys:

```
# app.yaml
application:
# llm.yaml
llm:
# embedding.yaml
embedding:
```

Therefore there is no conflict.

We end up with:

```
{
  "application": {...},
  "llm": {...},
  "embedding": {...},
  "vector_database": {...},
  "retrieval": {...}
}
```

8. Let's make the function more robust

Our current function works, but let's improve it.

```
def load_all_configs(config_dir):
    config_dir = Path(config_dir)

    if not config_dir.exists():
        raise FileNotFoundError(
            f"Configuration directory not found: {config_dir}"
        )

    configs = {}

    yaml_files = list(config_dir.glob("*.yaml"))

    if not yaml_files:
        raise FileNotFoundError(
            f"No YAML configuration files found in: {config_dir}"
        )

    for config_file in yaml_files:
        config = load_yaml(config_file)

        if not isinstance(config, dict):
            raise ValueError(
                f"Configuration must be a dictionary: {config_file}"
            )

        configs.update(config)

    return configs
```

Now we're doing validation:

```
config directory | ▼ Does it exist? / No Yes | | Error Find YAML | ▼ Any YAML files? / No Yes | |
Error Load YAML | ▼ Is it a dict? / No Yes | | Error Merge
```

This is much closer to production-quality code.

9. Add another YAML file

Now let's test the whole point of automation.

Create:

```
config/chunking.yaml
```

with:

```
chunking:
  strategy: recursive
  chunk_size: 1000
  chunk_overlap: 200
```

Don't modify `load_config.py`.

Run:

```
python src/load_config.py
```

The new configuration is automatically loaded.

Test it by adding:

```
print(config["chunking"]["strategy"])
print(config["chunking"]["chunk_size"])
print(config["chunking"]["chunk_overlap"])
```

You should get:

```
recursive
1000
200
```

That's the key benefit.

10. We have now built something important

Your architecture is becoming:

```
yaml-learning/
├── config/
│   ├── app.yaml
│   ├── llm.yaml
│   ├── embedding.yaml
│   ├── vector_database.yaml
│   ├── retrieval.yaml
│   └── chunking.yaml
└── src/
    ├── yaml_loader.py
    └── load_config.py
```

And:

config/*.yaml | ▼ load_all_configs() | ▼ merged dictionary |  ▼ ▼ ▼ LLM
Embeddings Retrieval

11. But there's an important limitation

Our current structure is:

```
config["llm"]["model"]
```

This works.

But imagine your Enterprise RAG grows to:

20 YAML files

100 configuration values

Your code becomes heavily dependent on string keys:

```
config["llm"]["model"]  
config["embedding"]["model"]  
config["retrieval"]["top_k"]  
config["chunking"]["chunk_size"]
```

There is no protection against:

```
config["llm"]["modle"]
```

Typo → runtime error.

And there's no guarantee that:

```
top_k: "five"
```

is actually an integer.

This is where typed configuration becomes valuable.

12. And this leads directly to our Enterprise RAG architecture

We're currently here:

YAML files | ▼ PyYAML loader | ▼ Python dictionary

Our eventual architecture will be:

YAML files | ▼ PyYAML loader | ▼ Configuration Manager | ▼ Pydantic Settings | Validation | ▼ Settings
Object |  ▼ ▼ ▼ RAG LLM Retrieval

Then your application code can eventually say:

```
settings.llm.model
```

instead of:


```
config["llm"]["model"]
```

And VS Code can provide autocomplete.

Your exercise before moving on

Add these two files:

config/chunking.yaml

```
chunking:
  strategy: recursive
  chunk_size: 1000
  chunk_overlap: 200
```

config/logging.yaml

```
logging:
  level: INFO
  file: logs/application.log
```

Do not change `load_all_configs()` to specifically mention either file.

Then print:

```
print(config["chunking"]["chunk_size"])
print(config["logging"]["level"])
```

Expected:

```
1000
INFO
```

If that works, you've successfully implemented automatic YAML configuration discovery.

Next major stage

We'll then leave the basic PyYAML mechanics and enter Stage 3 — YAML-Based Application Configuration.

That's where we'll take what you've built and start designing the actual architecture that will eventually replace the hardcoded configuration values in your current Enterprise RAG `settings.py`, while keeping `settings.py` as the clean configuration interface.